Platform	Wokwi.com (wokwi.com/projects/new/micropython-esp32)
Hardware	ESP32-S3-WROOM-1 (virtual simulation)
Language	MicroPython — Python jaise hi syntax!
Bhasha	Hinglish (Hindi + English)
Workshop	Smart Traffic System — Hardware Bridge Days

Activities in this PDF

Activity 1	■	**Single LED Blink**	GPIO basics, MicroPython intro
Activity 2	■	**Traffic Light Simulation**	3 LEDs — Red Yellow Green cycle
Activity 3	■	**Emergency Override**	Button press — ambulance simulation
Activity 4	■	**Dual Lane Competition**	2 students compete — busiest lane wins!

Workshop connection

Wokwi ESP32	**MicroPython Code**	**GPIO Logic**	**Jetson Nano**

Yeh 3 days hardware aane se pehle ka bridge hai — Wokwi pe jo seekhoge wahi Jetson pe lagega!

Teacher ke liye note

Har student wokwi.com/projects/new/micropython-esp32 pe naya project banaye.
main.py tab hona chahiye — sketch.ino nahi — warna MicroPython nahi chalega!
Har activity pair mein karo — ek type kare, ek guide kare, 15 min baad swap.
Activity 4 competition hai — class mein bahut energy aati hai iss mein!

Wokwi + ESP32 + MicroPython — Introduction

Wokwi kya hai?

Wokwi ek free online simulator hai — bina real hardware ke electronics simulate karo.
Browser mein khulta hai — koi install nahi, koi hardware nahi.
ESP32, Arduino, Raspberry Pi Pico — sab simulate hote hain.
Hum ESP32-S3 use karenge — bilkul waise jaise Jetson pe GPIO kaam karta hai!

Term	Matlab
`Microcontroller`	Ek chhota computer — sirf ek kaam karta hai, fast aur cheap (Arduino, ESP32)
`Single Board Computer`	Poora computer ek board pe — OS bhi chala sakta hai (Jetson Nano, Raspberry Pi)
`GPIO`	General Purpose Input Output — pins jo software se control hoti hain
`MicroPython`	Python ka chhota version — microcontrollers pe chalta hai

MicroPython vs Python — kya fark hai?

MicroPython bilkul Python jaisa hi hai — same syntax, same logic!
Sirf kuch libraries alag hain — machine, utime — baaki sab same.
Agar Python aata hai toh MicroPython 10 minute mein samajh aayega!

MicroPython Code	Kya karta hai?
`from machine import Pin`	ESP32 ke GPIO pins control karna
`Pin(2, Pin.OUT)`	GPIO 2 ko output mode mein set karo
`pin.value(1)`	Pin HIGH karo — LED ON
`pin.value(0)`	Pin LOW karo — LED OFF
`time.sleep(1)`	1 second ruko — same as Python!

Wokwi project kaise banate hain?

1. wokwi.com pe jao
2. 'New Project' click karo
3. 'MicroPython on ESP32' select karo
4. main.py tab dikhega — yahan code likhna hai
5. '+' button se components add karo — LED, Resistor, Button
6. Green Play button se simulate karo!

■ Activity 1

Single LED Blink — GPIO Basics

Real life analogy

Yeh wahi moment hai — pehli baar code likhke kuch physical cheez ko control karna.
Bilkul jaise Morse code mein signal bhejte hain — on/off, on/off.
Traffic system mein: yahi on/off signal traffic light ko control karega!

Circuit — Wokwi pe yeh add karo

ESP32 Pin	Connects to	Then to	Purpose
GPIO 2	Resistor (220Ω)	LED Anode (+)	*Signal — LED on/off control*
GND	LED Cathode (-)	—	*Circuit complete karna*

Resistor kyun zaroori hai?

LED directly connect karo toh burn ho jaati hai — bahut zyada current flow hoti hai.
220Ω resistor current ko limit karta hai — LED safe rehti hai.
Real Jetson GPIO pe bhi yahi rule — hamesha resistor lagao!

LED legs — dhyan se dekho!

Lambi leg = Anode = (+) = Resistor se connect karo
Chhoti leg = Cathode = (-) = GND se connect karo
Ulta connect kiya toh LED nahi jalegi — yeh ek common mistake hai!

Code — main.py mein likho

```
from machine import Pin

import time


# GPIO 2 ko output mode mein set karo

led = Pin(2, Pin.OUT)


print('LED Blink start!')
```

```
while True:

    led.value(1)        # LED ON

    print('ON')

    time.sleep(1)       # 1 second ruko


    led.value(0)        # LED OFF

    print('OFF')

    time.sleep(1)       # 1 second ruko
```

Output / Serial Monitor:

```
LED Blink start!
ON
OFF
ON
OFF
(LED har second blink karti rahegi...)
```

Code line by line samjho:

Code	Kya karta hai?
`from machine import Pin`	ESP32 ke pins control karne ka tool import karo
`Pin(2, Pin.OUT)`	GPIO 2 = output mode — hum signal bhejenge
`led.value(1)`	Pin HIGH = 3.3V = LED ON
`led.value(0)`	Pin LOW = 0V = LED OFF
`time.sleep(1)`	1 second wait — bilkul Python jaisa!

Jetson Nano se compare karo:

MicroPython aur Jetson.GPIO mein sirf library ka naam alag hai — logic same hai!

MicroPython (Wokwi)

```
from machine import Pin
led = Pin(2, Pin.OUT)
led.value(1)  # ON
led.value(0)  # OFF
```

Jetson.GPIO (Real hardware)

```
import Jetson.GPIO as GPIO
GPIO.setup(2, GPIO.OUT)
GPIO.output(2, GPIO.HIGH)
GPIO.output(2, GPIO.LOW)
```

Try it yourself:

LED blink karo — ON/OFF terminal mein dikh raha hai?

time.sleep(1) ko 0.2 karo — LED fast blink karti hai?

time.sleep(1) ko 3 karo — slow blink?

Challenge: 3 baar ON karo, phir 3 baar OFF — pattern create karo!

■ Activity 2

Traffic Light Simulation — Red Yellow Green

Real life analogy

Ab ek LED se teen LEDs — bilkul real traffic light jaise!
Green = gaadiyaan chal sakti hain (5 seconds)
Yellow = slow down karo, rukne wale hain (2 seconds)
Red = ruko! (5 seconds)
Yahi exact logic Jetson pe real traffic lights control karega!

Circuit — 3 LEDs add karo

ESP32 Pin	Connects to	Then to	Purpose
GPIO 4	Resistor (220Ω)	Green LED (+)	*Green signal control*
GPIO 5	Resistor (220Ω)	Yellow LED (+)	*Yellow signal control*
GPIO 2	Resistor (220Ω)	Red LED (+)	*Red signal control*
GND	All LED (-)	—	*Common ground*

Wokwi mein LED color kaise change karein?

LED pe click karo → left side mein Properties panel khulega
'color' field mein type karo: red / yellow / limegreen
Teenon LEDs alag alag color set karo — visually clear dikhega!

Code — main.py mein likho

```
from machine import Pin

import time


# ■■ Pins setup ■■■■■■■■■■■■■■■■■■■■■■■■

green  = Pin(4, Pin.OUT)

yellow = Pin(5, Pin.OUT)

red    = Pin(2, Pin.OUT)
```

```
# ■■ Sab band karo shuru mein ■■■■■■■■

green.value(0)

yellow.value(0)

red.value(0)


print('Traffic Light ready!')


# ■■ Traffic light cycle ■■■■■■■■■■■■■■

while True:


  # GREEN — 5 seconds

    print('GREEN  - Gaadiyaan chal sakti hain!')

    green.value(1)

    yellow.value(0)

    red.value(0)

    time.sleep(5)


  # YELLOW — 2 seconds

    print('YELLOW - Slow down karo!')

    green.value(0)

    yellow.value(1)

    red.value(0)

    time.sleep(2)


  # RED — 5 seconds

    print('RED    - Ruko!')
```

```
    green.value(0)

    yellow.value(0)

    red.value(1)

    time.sleep(5)
```

Output / Serial Monitor:

```
Traffic Light ready!
GREEN  - Gaadiyaan chal sakti hain!
YELLOW - Slow down karo!
RED    - Ruko!
GREEN  - Gaadiyaan chal sakti hain!
(cycle repeat hota rahega...)
```

Sab band kyun karte hain shuru mein?

green.value(0), yellow.value(0), red.value(0) — yeh startup safety hai.
Kabhi kabhi ESP32 reset hone pe pins random state mein hoti hain.
Sab zero karne se hum sure hain — ek hi LED on hogi ek time pe.
Real traffic systems mein yeh bahut important safety rule hai!

Workshop connection — Jetson pe yahi code:

time.sleep(5) → green_time = 10 + vehicle_count * 2 (Day 2 ka formula!)
Abhi hardcoded hai — baad mein YOLO count se automatically calculate hoga.
Yeh sirf 3 lines change karni hain jab Jetson aayega!

Try it yourself:

Traffic light chalao — teenon LEDs sahi sequence mein blink kar rahi hain?
Green ka time 3 seconds karo, Red ka 7 seconds — busy intersection simulate karo.
Challenge: Ek aur state add karo — 'All RED' 1 second ke liye Green se pehle.
Yeh pedestrian crossing signal jaise hoga!

■ Activity 3

Emergency Override — Ambulance Button

Real life analogy

Ambulance aa rahi hai — traffic light ka normal cycle rok ke immediately green karo!
Yahi emergency override hota hai — real traffic systems mein bhi hota hai.
Button = emergency vehicle sensor (real mein IR sensor ya GPS hota hai)
Button dabao = ambulance detected = immediately GREEN!

Circuit — Button add karo

ESP32 Pin	Connects to	Then to	Purpose
GPIO 4	Resistor (220Ω)	Green LED (+)	*Green signal*
GPIO 5	Resistor (220Ω)	Yellow LED (+)	*Yellow signal*
GPIO 2	Resistor (220Ω)	Red LED (+)	*Red signal*
GPIO 15	Button pin 1	Button pin 2 → GND	*Emergency button*

Button Wokwi mein kaise add karein?

'+' button → search 'Pushbutton' → canvas pe drag karo
Button ke ek pin ko GPIO 15 se connect karo
Button ke doosre pin ko GND se connect karo
Pull-up resistor internal use karenge — baahri resistor nahi chahiye button ke liye!

Code — main.py mein likho

```
from machine import Pin

import time


# ■■ Pins setup ■■■■■■■■■■■■■■■■■■■■■■■■

green     = Pin(4,  Pin.OUT)

yellow    = Pin(5,  Pin.OUT)

red       = Pin(2,  Pin.OUT)

emergency = Pin(15, Pin.IN, Pin.PULL_UP)  # Button with pull-up
```

```
def all_off():
    green.value(0)
    yellow.value(0)
    red.value(0)

def emergency_green():
    print('EMERGENCY! Ambulance detected — GREEN override!')
    all_off()
    green.value(1)
    time.sleep(5)        # 5 seconds green for ambulance
    all_off()

all_off()
print('Traffic Light ready! Button = Emergency override')

while True:
  # Emergency check — pehle dekho
    if emergency.value() == 0:   # Button dabaya = 0 (PULL_UP ki wajah se)
        emergency_green()
        continue                  # Cycle restart karo

  # Normal GREEN
    print('GREEN  - Normal cycle')
    all_off()
    green.value(1)
```

```
    for _ in range(50):          # 5 seconds, har 0.1s check karo
        if emergency.value() == 0:
            emergency_green()
            break
        time.sleep(0.1)

# Normal YELLOW
    print('YELLOW - Slow down')
    all_off()
    yellow.value(1)
    time.sleep(2)

# Normal RED
    print('RED    - Stop')
    all_off()
    red.value(1)
    for _ in range(50):          # 5 seconds, har 0.1s check karo
        if emergency.value() == 0:
            emergency_green()
            break
        time.sleep(0.1)
```

Output / Serial Monitor:

```
Traffic Light ready! Button = Emergency override
GREEN  - Normal cycle
YELLOW - Slow down
RED    - Stop
EMERGENCY! Ambulance detected — GREEN override!
GREEN  - Normal cycle
(button dabate hi override hota hai...)
```

Pin.PULL_UP kya hota hai?

Button directly connect karne pe pin 'floating' hoti hai — random values aate hain.
PULL_UP = internally 3.3V se connect karo — button nahi dabaya toh value = 1
Button dabao toh GND se connect hota hai — value = 0
Isliye code mein if emergency.value() == 0 likhte hain — dabaya matlab 0!

Loop mein emergency check kyun?

Agar sirf while loop ke shuru mein check karein — RED ke beech button dabao toh miss ho jaayega.
Isliye har phase mein 0.1s wale loop mein bhi check karte hain.
Yahi 'interrupt-like behavior' hai — real systems mein actual hardware interrupts use hote hain.

Try it yourself:

Normal cycle chalao — phir button dabao — immediately green hua?
RED phase mein button dabao — override kaam karta hai?
Challenge: Emergency ke baad 2 baar Red do — 'clear the intersection' logic.
emergency_green() function ke baad red.value(1) + time.sleep(2) add karo.

■ Activity 4

Dual Lane Competition — Busiest Lane Wins!

Real life analogy

2 lanes hain — North aur South.
Dono lanes mein vehicles aa rahe hain — jis lane mein zyada vehicles, use green milega!
Button A = North lane mein ek vehicle aaya
Button B = South lane mein ek vehicle aaya
Har 10 seconds mein count compare karo — winner ko green do!
Yahi exact logic hai jo YOLO ke saath real system mein chalega!

Circuit — 2 Buttons + 2 LED sets

ESP32 Pin	Connects to	Then to	Purpose
GPIO 4	Resistor (220Ω)	Green LED 1 (+)	*North lane green*
GPIO 2	Resistor (220Ω)	Red LED 1 (+)	*North lane red*
GPIO 18	Resistor (220Ω)	Green LED 2 (+)	*South lane green*
GPIO 19	Resistor (220Ω)	Red LED 2 (+)	*South lane red*
GPIO 15	Button A pin 1	GND	*North lane vehicle button*
GPIO 16	Button B pin 1	GND	*South lane vehicle button*

Code — main.py mein likho

```
from machine import Pin

import time


# ■■ Lane 1 (North) pins ■■■■■■■■■■■■■■

north_green = Pin(4,  Pin.OUT)

north_red   = Pin(2,  Pin.OUT)

btn_north   = Pin(15, Pin.IN, Pin.PULL_UP)


# ■■ Lane 2 (South) pins ■■■■■■■■■■■■■■
```

```
south_green = Pin(18, Pin.OUT)

south_red   = Pin(19, Pin.OUT)

btn_south   = Pin(16, Pin.IN, Pin.PULL_UP)


# ■■ Vehicle counters ■■■■■■■■■■■■■■■■■■

north_count = 0

south_count = 0


def set_signals(north_wins):

    if north_wins:

        north_green.value(1); north_red.value(0)

        south_green.value(0); south_red.value(1)

        print(f'NORTH GREEN! ({north_count} vehicles)')

    else:

        north_green.value(0); north_red.value(1)

        south_green.value(1); south_red.value(0)

        print(f'SOUTH GREEN! ({south_count} vehicles)')


print('Competition start! 10 seconds mein zyada button dabao!')

print('Button A = North lane | Button B = South lane')


while True:

    north_count = 0

    south_count = 0

    print('--- New Round --- 10 seconds!')
```

```
# ■■ 10 second counting window ■■■■
  start = time.time()
  while time.time() - start < 10:
      if btn_north.value() == 0:
          north_count += 1
          print(f'North: {north_count}')
          time.sleep(0.2)      # Debounce
      if btn_south.value() == 0:
          south_count += 1
          print(f'South: {south_count}')
          time.sleep(0.2)      # Debounce

# ■■ Winner decide karo ■■■■■■■■■■■■
  print(f'Result — North: {north_count} | South: {south_count}')
  if north_count >= south_count:
      set_signals(north_wins=True)
  else:
      set_signals(north_wins=False)

  time.sleep(5)    # 5 second green time, phir naya round
```

Output / Serial Monitor:

```
Competition start! 10 seconds mein zyada button dabao!
Button A = North lane | Button B = South lane
--- New Round --- 10 seconds!
North: 1
North: 2
South: 1
North: 3
South: 2
Result — North: 3 | South: 2
NORTH GREEN! (3 vehicles)
--- New Round --- 10 seconds!
```

Debounce kya hota hai?

Button dabane pe kabhi kabhi ek press = multiple signals — mechanical bounce.
time.sleep(0.2) = 200ms wait — ek press sirf ek baar count ho.
Real systems mein hardware debounce circuits ya software filtering hoti hai.

YOLO se connection — yeh exactly wahi logic hai!

Button press = YOLO ne ek vehicle detect kiya
north_count = lane_counts['North'] (Day 6 mein seekhenge)
set_signals() = GPIO traffic light control (Day 9-13 mein Jetson pe)
10 sec window = detection interval (real mein har frame pe hoga)
Yeh competition actually poore Smart Traffic System ka prototype hai!

Try it yourself:

2 students pair mein — ek North button, ek South button dabao.
10 seconds mein zyada press karo — tumhari lane green hogi!
Challenge: 4 lanes banao — GPIO 4,5,2,18 pe 4 green LEDs.
4 buttons — jis lane mein sabse zyada count — sirf us lane ko green do.
Hint: max() function use karo — Day 2 mein padhaya tha!

Poora System — Ab Sab Connect Karo!

Wokwi se Jetson tak — yeh hai aapka journey:

Wokwi Activity 1: LED blink → GPIO basics samjhe
Wokwi Activity 2: Traffic light → Signal cycle samjha
Wokwi Activity 3: Emergency → Override logic samjha
Wokwi Activity 4: Competition → Vehicle count → signal logic samjha
Jab Jetson aayega: Sirf 4 lines change karni hain — baaki sab same!

MicroPython (Wokwi)

```
# MicroPython (Wokwi)
from machine import Pin
green = Pin(4, Pin.OUT)
green.value(1)   # ON
green.value(0)   # OFF
btn = Pin(15, Pin.IN,
         Pin.PULL_UP)
if btn.value() == 0:
    emergency_green()
```

Jetson.GPIO (Real hardware)

```
# Jetson.GPIO
import Jetson.GPIO as GPIO
GPIO.setup(4, GPIO.OUT)
GPIO.output(4, GPIO.HIGH)
GPIO.output(4, GPIO.LOW)
GPIO.setup(15, GPIO.IN,
  GPIO.PUD_UP)
if GPIO.input(15) == 0:
    emergency_green()
```

Congratulations!

Aapne bina real hardware ke poora traffic light system simulate kar liya!

LED blink se lekar vehicle counting competition tak — yeh sab Jetson pe ek din mein kaam karega.

Hardware aane pe sirf library swap karo — logic bilkul same rehega.

Yahi hai software + hardware ka real world connection!